Ders 16 Çalışma Özeti

Ders 16 Çalışma Özeti

Genel Konular

- Dersin tarihi 5 Mayıs 2021 Çarşamba. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:03'te başlar. Ara yaklaşık 01:04-01:13 ve 01:58-02:07 arasındadır.
- Duyurular
  - Gelecek hafta idari izin nedeniyle Pazartesi-Cuma bütün dersler iptal. Laboratuvar da yapılamayacak. Sonraki hafta 19 Mayıs tatili. İki hafta ders yok.
  - Bu akşamki uygulama saat 19:00'da Toprak Hoca tarafından yapılacak. İftara denk gelenler kaydı sonra izleyebilir.
  - Yapılamayan laboratuvar (Lab 3) bir hafta sonu gününe alınacak veya ertelenecek. Kalan laboratuvarlar 26 Mayıs ve 9 Haziran.
  - Sınav kâğıdı itirazları için dersin sonunda ayrı bir Zoom linki açıldı.
- Vize çözümleri ve değerlendirme
  - Ortalama 67. Yaklaşık 5-6 kişi sınava girmedi.
  - En çok puan 5. soruda (sekans diyagramı) ve 4. sorunun bir kısmında (`Actor` sınıfı) kaybedildi.
  - 1. soru: Soyut `Person` sınıfı (constructor ve `getName`).
  - 2. soru: 3 metotlu `Act` interface'i.
  - 3. soru: `Movie` sınıfı `Act`'i implement eder. Constructor, IMDb puanı için get/set, `getName`, `getMoreInfo`.
  - 4. soru: `Actor` sınıfı. `Act` türünde referanslar tutan bir dizi, `addAct` metodu ve `toString`.
  - 5. soru: `toString` metodunun sekans diyagramı.
  - 6. soru: `main` içinde nesneler üretip `toString` çağırmak.
- Map ve hashing
  - Hash veri yapısı, bir elemana ortalamada sabit zamanda ($O(1)$) erişmeyi sağlar. Konum bir hash fonksiyonuyla bulunur. Basit örnek: `anahtar mod dizi_boyutu`.
  - Çakışma (collision) olursa eleman bir sonraki boş yere konur. Dizi dairesel (circular) düşünülür.
  - Java'da `Map` interface'i, `AbstractMap` ve `HashMap` sınıfları vardır. `HashMap` thread-safe değildir, `Hashtable` thread-safe'tir.
  - `HashMap<K, V>`: `K` anahtar, `V` değer tipidir. Metotlar: `get(key)`, `put(key, value)`, `values()`, `size()`.
  - `put` aynı anahtara yeni değer yazarsa eski değeri döndürür ve yerine yenisini koyar.
  - Örnek: `Course` sınıfı öğrencileri `HashMap<String, Student>` içinde tutar. Anahtar öğrenci numarasıdır, çünkü benzersizdir.
  - Değer olarak bir liste de tutulabilir: `HashMap<String, ArrayList<Student>>`. Ders kodu anahtar, dersi alan öğrenciler değer olur. Bu basit bir veritabanı gibi çalışır.
- Java Collection Framework
  - Veri yapıları dersinde öğrenilen yapıları ve algoritmaları (sıralama, arama) hazır paket olarak sunar. Nesne yönelimli bir hiyerarşiyle yazılmıştır.
  - Hiyerarşi: `Iterable` → `Collection` → `List`, `Queue`, `Set`. Sarı kutular interface, kırmızı kutular sınıftır.
  - `Set`'in farkı: Aynı eleman iki kez eklenemez.
  - `Collection` metotları: `add`, `addAll`, `remove`, `removeAll`, `retainAll`, `size`, `clear`, `contains`, `containsAll`, `iterator`, `toArray`, `isEmpty`, `equals`, `hashCode`.

  - `Iterator`: `hasNext()` ve `next()` ile listede gezinilir.
  - `ArrayList` ek metotları: `lastIndexOf`, `indexOf`, `trimToSize`.
  - `LinkedList` Java'da çift bağlı listedir. `addFirst`, `addLast`, `removeFirst`, `getFirst`, `getLast` metotlarıyla stack ve kuyruk (queue) kurulabilir.
  - `ArrayList` ile `LinkedList` farkı: `ArrayList` dinamik dizi kullanır, erişim hızlı ama ekleme ve silmede kaydırma gerekir. `LinkedList` referanslarla bağlanır, ekleme ve silme hızlıdır. `LinkedList` hem `List` hem kuyruk olarak davranabilir.
- İstisna yönetimi (exception handling)
  - Hatadan kaçınma (avoidance) ile hata yönetimi farklıdır. Kaçınma: `if` ile girdiyi kontrol etmek (ör. 0-100 arası sayı). İstisna yönetimi: Hata oldu, şimdi programın çökmemesi için ne yapacağız?
  - Sınıf hiyerarşisi: `Throwable` → `Error` ve `Exception`. `Exception` altında `IOException`, `RuntimeException` vb.
  - `Error` örnekleri: Sistem kaynaklarının tükenmesi, JVM hataları, desteklenmeyen sınıf sürümü.
  - Çalışma zamanı istisnaları: `ArithmeticException` (sıfıra bölme), `ArrayIndexOutOfBoundsException`, `NullPointerException` (nesne atanmamış referans), `NumberFormatException` (`"abc"`'yi sayıya çevirmek).
  - 5 anahtar kelime: `try`, `catch`, `finally`, `throw`, `throws`.
  - Kendi istisna sınıfımızı `Exception`'dan (veya `IOException` gibi bir alt sınıftan) kalıtımla yazabiliriz. Constructor'da mesaj `super(mesaj)` ile üst sınıfa gönderilir.
  - `final`, `finally` ve `finalize` farklı şeylerdir.
  - Override edilen metotlarda istisna bildirme kuralları gösterildi.
  - `System.err.println` hata akışına yazar. IDE'de kırmızı görünür.

Hocanın Özellikle Vurguladığı Kısımlar

- **UML'in dışına çıkmayın.** UML'de olmayan bir metot (ör. `toString`) eklemek puan kaybettirdi.
  - Gizli bilgi (hidden information) yalnızca üye alanlarda olur. Gizli metot diye bir şey yok.
- **Nesnelerin benzersizliğini isimle değil, referansla kontrol edin.**
  - `addAct` içinde "bu nesne daha önce eklenmiş mi?" diye bakarken == ile nesne referansları karşılaştırılmalı.
  - İsim ilkel bir veridir, nesneyi temsil etmez. Aynı isimde iki kişi olabilir. İsimle karşılaştıranlardan puan kırıldı.
- **Algoritmik verimlilik de değerlendirildi.** Diziye eleman eklerken iç içe iki döngü kullananlardan puan kırıldı. Diziyi iki kez gezmenin anlamı yok.
- Nesneye ait dizi, nesne oluşturulurken (constructor'da veya alan tanımında) oluşturulmalı. Diziyi `static` yapmak veya yanlış yerde `new` ile oluşturmak puan kaybettirdi.
- `super(...)` ile üst sınıf constructor'ını çağırmayanlardan puan kırıldı.
- Sekans diyagramında `info` değişkeni hemen oluşmaz. Arada `getName`, `getStageName`, `getYear`, `getMoreInfo` çağrıları vardır. Bunlar diyagramda gösterilmeli.
- **Bir sınıf soyutsa soyut metot içermek zorunda değildir. Ama soyut metot içeriyorsa soyut olmak zorundadır.**
- **Interface içine metot gövdesi yazılmaz.**
- `HashMap` konusu önemli. Veritabanı tarzı uygulamalar (benzersiz anahtar ve kayıt bilgisi) bununla kolayca yazılır.
- `try` ve `catch` birbirinden ayrılmaz. `try` yazıp `catch` (veya `finally`) yazmazsanız derleme hatası alırsınız.
- Birden fazla `catch` yazarken özelden genele doğru sıralayın.
- `throw` ile `throws`'u karıştırmayın. `throws` metot imzasında "fırlatabilir" diye bildirir. `throw` istisnayı gerçekten fırlatır ve `new` ile kullanılır.
- Dersler sırasında soru gelmiyor, sorular sınavdan sonra geliyor. Hoca soruların ders sırasında sorulmasını istedi.

Kısa Tekrar Notları

- Hash: konum = hash(anahtar). Ortalama erişim $O(1)$. Çakışmada sonraki boş yere (doğrusal yoklama).
- `HashMap<K,V>`: `put`, `get`, `values`, `size`. Anahtar benzersiz olmalı.
- `HashMap` thread-safe değil, `Hashtable` thread-safe.
- `Set` tekrar eden eleman kabul etmez.
- `removeAll(c)`: c'dekileri siler. `retainAll(c)`: yalnızca c'dekileri bırakır.
- `LinkedList` Java'da çift bağlıdır.
- `try` → hata olursa ilgili `catch` → her durumda `finally`.
- `catch` sırası: özel istisna önce, `Exception` en sonda.
- `throws`: metot bildirimi, birden fazla istisna yazılabilir. `throw new X(...)`: tek bir istisna fırlatır.
- Checked istisna (ör. `IOException`, kendi `Exception` alt sınıfınız): `try-catch` içinde yakalanmalı veya `throws` ile bildirilmeli. Unchecked (`RuntimeException` ve alt sınıfları): zorunlu değil.
- Özel istisna: `class InvalidAgeException extends Exception { InvalidAgeException(String s) { super(s); } }`.
- `final`: sabit. `finally`: her durumda çalışan blok. `finalize`: çöp toplayıcının nesneyi silerken çağırdığı metot.

Detaylı Açıklamalar

Vize cevap anahtarı (derste ekranda gösterilen belgeden)

Puanlar: 1. soru 10, 2. soru 10, 3. soru 20, 4. soru 30, 5. soru 15.

```java
public abstract class Person {
    private String name;
    public Person(String name) { this.name = name; }
    public String getName() { return name; }
}

public interface Act {
    public String getName();
    public int getYear();
    public String getMoreInfo();
}

public class Movie implements Act {
    private String movieName;
    private int year;
    private double imdbScore;

    public Movie(String name, int year) {
        this.movieName = name;
        this.year = year;
    }
    public double getImdbScore() { return imdbScore; }
    public void setImdbScore(double score) { this.imdbScore = score; }

    public String getName() { return movieName; }
    public int getYear() { return year; }
    public String getMoreInfo() { return "(imdb: " + imdbScore + ")"; }
}

public class Actor extends Person {
    private String stageName;
    private Act acts[];
    private final static int maxAct = 50;
    private int actCount = 0;
```

```
    public Actor(String name, String stageName) {
        super(name);
        this.stageName = stageName;
        acts = new Act[maxAct];          // nesneye ait dizi, constructor'da oluşturulur
    }
    public String getStageName() { return stageName; }
    public void setStageName(String stageName) { this.stageName = stageName; }
    public void addAct(Act anAct) {
        for (int i = 0; i < actCount; i++)
            if (anAct == acts[i])          // referans karşılaştırması
                return;
        if (actCount < maxAct) {
            acts[actCount] = anAct;
            actCount++;
        }
    }
    public String toString() {
        String info;
        info = "My name is " + getName();
        info += " and my stage name is " + stageName + ". ";
        info += "My acts are: ";
        for (int i = 0; i < actCount; i++)
            info += " " + acts[i].getName() + "(" + acts[i].getYear() + ")" + acts[i].getMoreI
        return info;
    }
}
```

- Interface türünde referans tutan bir dizi tanımlamak mümkündür. Diziye `Movie` gibi `Act`'i implement eden nesneler konur.
- `maxAct`'in `static` olması zorunlu değil. Hoca sayacı başka şekilde tutan çözümleri de kabul etti. Puan kırılan nokta dizinin kendisinin (`acts`) yanlış yerde oluşturulmasıydı.
- `toString` içindeki aktör bilgileri için ayrı bir private yardımcı metot yazanların çözümü de kabul edildi.
- İçeriği karşılaştırmak isteseydiniz alanları tek tek karşılaştırmanız gerekirdi. Bu iyi bir çözüm değildir. Burada nesne benzersiz olmalıdır.

Hash örneği

Boyutu 7 olan bir dizi, `konum = anahtar mod 7`:

- 17 → 3, 5 → 5.
- 12 → 5. Konum dolu, çakışma var. Sonraki boş yere, 6'ya konur.
- 13 → 6. Konum dolu. Dizinin sonuna gelindiği için başa dönülür ve 0'a konur.
- 12 aranırken önce 5'e bakılır. Orada 5 vardır, 12 değil. Sonraki konuma (6) geçilir ve 12 bulunur.

`HashMap` ile kurs ve öğrenciler

```
public class Course {
    private String code, name;
    private int capacity;
    private HashMap<String, Student> students;

    public Course(String code, String name, int capacity) {
        this.code = code; this.name = name; this.capacity = capacity;
        students = new HashMap<String, Student>();
    }
    public int getStudentCount() { return students.size(); }
    public boolean addStudent(Student s) {
        if (getStudentCount() == capacity || findStudent(s.getNumber()) != null) return false;
```

```
        students.put(s.getNumber(), s);   // anahtar: öğrenci numarası
        return true;
    }
    public Student findStudent(String number) { return students.get(number); } // yoksa null
    public void showClassList() {
        for (Student s : students.values())
            System.out.println(s.getNumber() + " " + s.getName());
    }
}
```

İstisna yönetimi örnekleri

```
int[] a = new int[5];
try {
    a[5] = 30 / 0;      // önce bölme yapılır ve ArithmeticException fırlatılır
} catch (ArithmeticException e) {
    System.out.println("Aritmetik: " + e.getMessage());   // "/ by zero"
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Dizi: " + e.getMessage());
} finally {
    System.out.println("Her durumda calisir");   // dosya/veritabanı kapatma gibi işler
}
System.out.println("rest of the code");
```

- `30 / 1` yazılsaydı bölme başarılı olur, `a[5]` ataması `ArrayIndexOutOfBoundsException` fırlatırdı.
- `e.printStackTrace()` istisnanın hangi satırlardan geçtiğini yazar (loglama için).
- `finally` üç durumda da çalışır: istisna yok; istisna var ve yakalandı; istisna var ama yakalanmadı. Son durumda `finally`'den sonra program sonlanır ve "rest of the code" görülmez.
- `try` blokları iç içe yazılabilir.

Kendi istisna sınıfımız ve `throw`/`throws`

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String s) { super(s); }
}

public class TestCustomException {
    static void validate(int age) throws InvalidAgeException {
        if (age < 18) throw new InvalidAgeException("not valid");
        System.out.println("welcome to vote");
    }
    public static void main(String[] args) {
        try {
            validate(13);
        } catch (InvalidAgeException e) {
            System.out.println(e);   // InvalidAgeException: not valid
        }
        System.out.println("rest of the code");
    }
}
```

- `validate(13)` `try` dışında çağrılsaydı ve istisna `ArithmeticException` olsaydı program "not valid" mesajıyla çökerdi. "rest of the code" yazılmazdı.
- `InvalidAgeException`, `Exception`'dan türediği için checked'tir. Çağıran kod onu yakalamak veya `throws` ile bildirmek zorundadır.

Override ve istisnalar

- Üst sınıftaki metot hiçbir istisna bildirmiyorsa, alt sınıftaki override edilen metot checked istisna (ör. `IOException`) bildiremez. Unchecked istisna (ör. `RuntimeException`, `ArithmeticException`) bildirebilir.
- Üst sınıftaki metot bir istisna bildiriyorsa, alt sınıftaki metot aynısını, onun alt sınıfını veya hiç istisna bildirmeyebilir. Daha genel bir istisna (üst sınıfını) bildiremez.
- `throw` ifadesinden sonra koşulsuz yazılan satırlar "unreachable statement" hatası verir. Derste `if` ile koşul eklenerek bu hata giderildi.
- Override edilen metotta `super.msg()` metodun herhangi bir yerinde çağrılabilir. Bu sıralama zorunluluğu yalnızca constructor'daki `super(...)` için geçerlidir.

Ders sonrası notlar

- **For-each ve iterator ilişkisi ters anlatıldı.** Bir öğrenci "for-each döngüsü arka planda iterator mı kullanıyor?" diye sordu. Hoca "tam tersi, iterator kullandığınızda for-each kullanıyor" dedi. Doğrusu öğrencinin söylediğidir: `Iterable` bir koleksiyon üzerindeki for-each döngüsü, derleyici tarafından `iterator()`, `hasNext()` ve `next()` çağrılarına çevrilir.
- **Hash erişimi "polinom olmayan zaman" değil, sabit zamandır.** Slayt anlatımında "non-polynomial time" dendi. Kastedilen, ortalama $O(1)$'dir. "$O(2)$" diye ayrı bir karmaşıklık sınıfı yoktur, o da $O(1)$'dir. Çok fazla çakışma olursa en kötü durum $O(n)$'e çıkabilir.
- **`IOException` neden yakalanamadı?** Hoca `5 / 0` içeren bir `try` bloğuna `catch (IOException e)` ekleyince hata aldı ve bunu "RuntimeException kapsadığı için" diye açıkladı. Asıl sebep: `IOException` checked bir istisnadır ve `try` bloğunda onu fırlatabilecek bir ifade yoktur. Derleyici "exception IOException is never thrown in body of corresponding try statement" hatası verir (Java ile doğrulandı).
- **`catch (Exception e)`'yi alt sınıflardan önce yazmak** derleme hatası verir: "exception ArithmeticException has already been caught" (Java ile doğrulandı).
- **"Custom exception yazıyorsak `try-catch` zorunludur" ifadesi eksik.** Zorunluluk, sınıf `Exception`'dan türeyip checked olduğunda vardır. `RuntimeException`'dan türetilirse zorunlu değildir. Ayrıca `try-catch` yerine çağıran metotta `throws` ile bildirmek de yeterlidir.
- **`main` metodunun `throws IOException` örneği:** Hoca "IOException zorunlu olmadığı için try bloğuna gerek yok" dedi. `IOException` checked'tir. Burada `try` gerekmemesinin sebebi `main`'in `throws IOException` ile bildirilmiş olmasıdır.
- **`finalize` eskimiştir.** Java 9'da deprecated oldu, Java 18'de kaldırılmak üzere işaretlendi. Kaynak bırakmak için `try-with-resources` ve `AutoCloseable` kullanılır.
- **Hoca birden fazla `catch` için "özelden genele yazmak gerekiyor diye biliyorum" dedi.** Bu doğrudur ve yukarıdaki derleme hatasıyla doğrulandı.
- **`retainAll` örneği:** `[1, 2, 3, 4]` listesinde `retainAll([2, 3])` sonucu `[2, 3]` olur. `put` örneği: Boş haritada `put("k","v1")` `null`, ardından `put("k","v2")` `"v1"` döndürür (Java ile doğrulandı).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.